NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DUARTE JOSÉ LOURENÇO BELO

Master in Computer Science and Engineering

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

MASTER IN COMPUTER SCIENCE AND ENGINEERING

SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon

Draft: April 29, 2026

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

DUARTE JOSÉ LOURENÇO BELO

Master in Computer Science and Engineering

Supervisor: Adviser Name
Full Professor, NOVA School of Science and Technology

MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
Draft: *April 29, 2026*

Abstract

Keywords: Multi-Cloud · Generative AI · Cost Optimisation · Cloud Operations · GenAI Frameworks

Resumo

Palavras-chave: Multi-Cloud · Inteligência Artificial Generativa · Otimização de Custos · Operações em Nuvem · Frameworks GenAI

Disclosure of Delegation to Generative Artificial Intelligence

Duarte José Lourenço Belo declares[1] the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [22], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- ☑ Idea generation
- ☑ Defining the research objective
- ☑ Formulating research questions and hypotheses
- ☐ Feasibility assessment and risk evaluation
- ☐ Preliminary hypothesis testing

Literature Review

- ☑ Literature search and systematization
- ☑ Writing the literature review
- ☐ Analysis of market trends and/or patent environment
- ☐ Evaluation of novelty and identification of gaps

Methodology

- ☑ Research design
- ☑ Development of experimental or research protocols
- ☐ Selection of research methods

Software Development and Automation

- ☑ Code generation
- ☑ Code optimization
- ☐ Process automation
- ☐ Creation of algorithms for data analysis

Data Management

- ☑ Data collection
- ☑ Validation
- ☑ Data cleaning
- ☐ Data curation and organization
- ☐ Data analysis
- ☐ Visualization
- ☐ Reproducibility testing

[1] This declaration was generated using LuaLaTeX [23] and the LaTeX package `aidisclose` [14].

Writing and Editing

- [x] Text generation
- [x] Proofreading and editing
- [x] Summarizing text
- [ ] Formulation of conclusions
- [ ] Adapting and adjusting emotional tone
- [ ] Translation
- [ ] Reformatting
- [ ] Press releases and outreach materials

Ethics Review

- [x] Bias analysis and discrimination assessment
- [x] Ethical risk analysis
- [x] Monitoring compliance with ethical standards
- [ ] Data confidentiality monitoring

Supervision

- [ ] Quality assessment
- [ ] Trend identification
- [ ] Identification of limitations
- [ ] Recommendations
- [ ] Publication support

Generative AI tools used: ChatGPT-5.2, Gemini 3 Pro, Claude 4.5 Opus, and Perplexity 12.12.25.

Additional comment #1: The AI was used primarily for refining the code in Section 3, but the algorithm logic was derived manually.

Additional comment #2: This is a second additional comment!

Additional comment: This is a third additional comment! It is starred and, thus, does not get a number. Also, this comment is longer and, hopefully, will span to the next line!

Additional comment #3: This is the third non-starred additional comment!

Contents

List of Figures

List of Tables

Listings

1

INTRODUCTION

1.1 Context and Motivation

1.2 Problem Statement

1.3 Research Objectives

1.4 Contributions

1.5 Document Structure

2

Background

2.1 Foundations of Neural Language Representation

The ability to represent language in a form that machines can process and reason over is a prerequisite for any modern Generative AI system. Two landmark contributions established the theoretical and practical foundations upon which contemporary large language models (LLMs) and retrieval-augmented architectures are built: the introduction of dense word representations via neural networks, and the development of the Transformer architecture based solely on attention mechanisms.

2.1.1 Distributed Word Representations

Prior to 2013, natural language processing relied predominantly on sparse, high-dimensional representations such as one-hot encodings, which offered no intrinsic notion of semantic similarity between words. Mikolov et al. [15] addressed this limitation by proposing Word2Vec, a computationally efficient framework for learning continuous vector representations of words from large unlabelled corpora. The model is trained through a self-supervised prediction task using one of two architectures: Continuous Bag-of-Words (CBOW), which predicts a target word from its surrounding context, or Skip-gram, which performs the inverse prediction.

The resulting word embeddings encode both syntactic and semantic structure in a dense vector space, enabling linear algebraic operations over meaning - most notably the well-known relationship $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$.

The significance of Word2Vec extends well beyond the paper itself. By demonstrating that semantic similarity can be captured as geometric proximity in a continuous vector space, it established the distributional hypothesis as an actionable engineering principle. This insight directly underlies the design of modern embedding models and vector databases, which are central components in Retrieval-Augmented Generation (RAG) pipelines [12]. In the context of this work, the quality and cost of the embedding stage in the Maestro framework's RAG pipeline is a direct consequence of choices made in this representational paradigm.

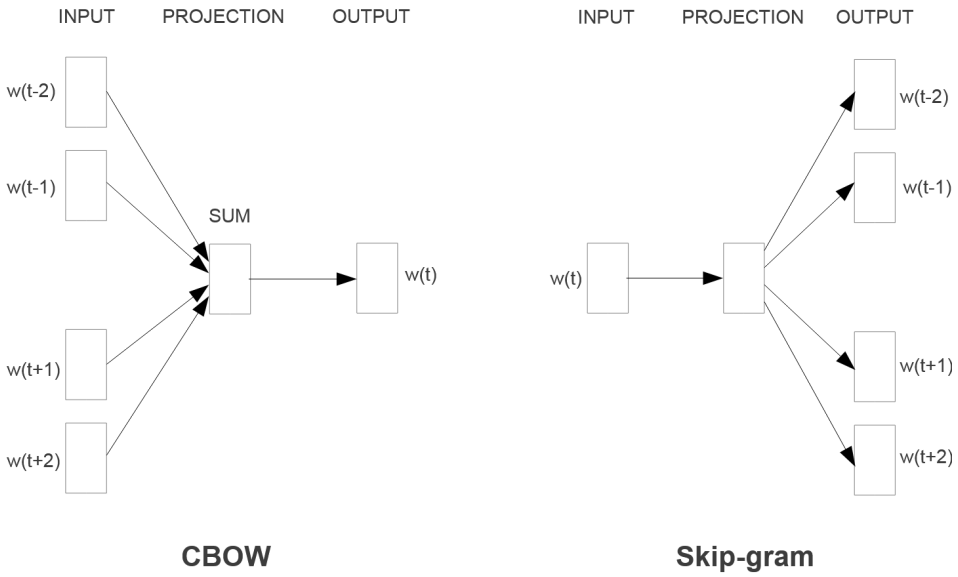

Figure 2.1: CBOW and Skip-gram architectures of the Word2Vec framework. Adapted from [15].

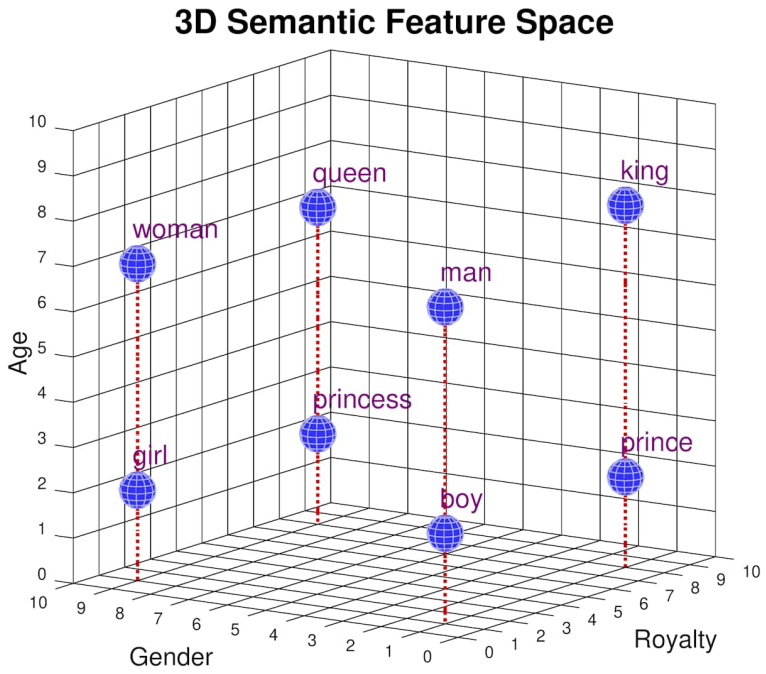

Figure 2.2: Word vector space learned by Word2Vec, illustrating semantic analogies such as $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$. Adapted from [15].

2.1.2 The Transformer Architecture

Despite the advances in distributed representations, sequence modelling remained constrained by recurrent architectures such as LSTMs and GRUs, which process tokens sequentially and suffer from vanishing gradients over long contexts. Vaswani et al. [25] proposed a radical departure: the Transformer, an architecture that dispenses with recurrence entirely and relies solely on attention mechanisms to model dependencies between all positions in a sequence simultaneously. The core operation, scaled dot-product attention, computes output representations as a weighted sum of value vectors, where weights are derived from the compatibility between query and key projections:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

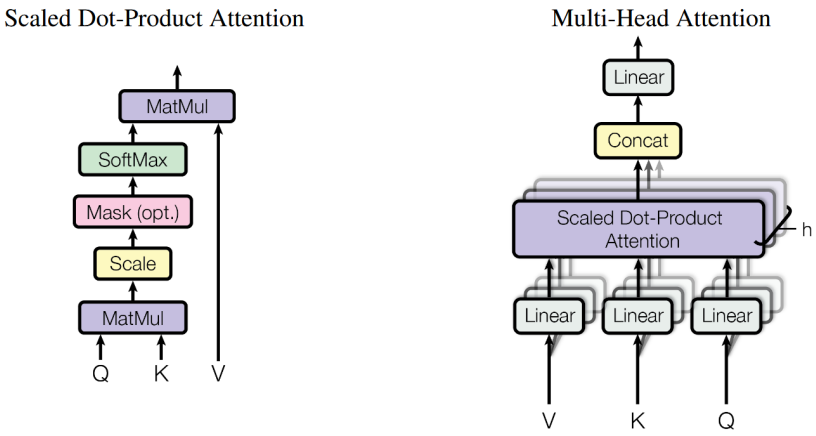

Figure 2.3: Scaled Dot-Product Attention (left) and Multi-Head Attention (right). Adapted from [25].

By running multiple such attention operations in parallel - Multi-Head Attention - the model learns to simultaneously attend to different types of relationships across the input, such as syntactic dependencies and semantic co-references. Positional encodings are injected additively into the input embeddings to compensate for the architecture's inherent permutation invariance. The resulting model achieved 28.4 BLEU on the WMT 2014 English-to-German translation benchmark, surpassing all prior approaches while requiring a fraction of the training compute [25].

The Transformer's parallelism over sequence positions unlocked a new scaling regime for neural language models. Architectures derived from it - most notably encoder-only models such as BERT [7] for representation learning, and decoder-only models such as the GPT series [4] for text generation - became the standard building blocks of enterprise GenAI systems. Critically, the attention mechanism scales quadratically with sequence length, $O(n^2)$, meaning that context length is a primary cost driver in token-priced LLM APIs. This architectural property is foundational to the FinOps strategies explored in this

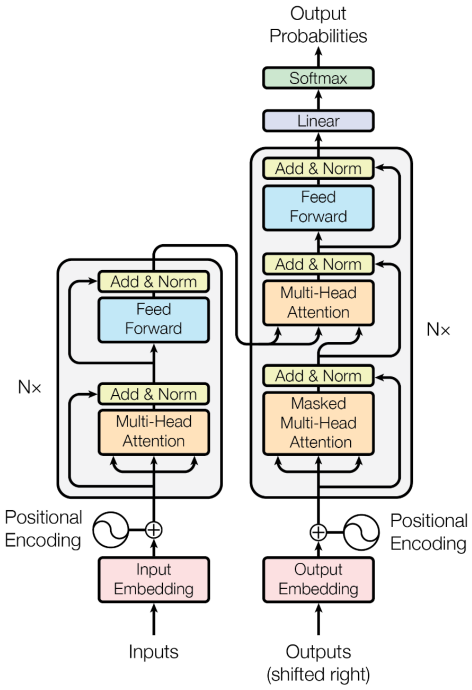

Figure 2.4: Full Transformer architecture with encoder and decoder stacks. Adapted from [25].

dissertation, namely prompt compression, semantic caching, and dynamic model selection, as each seeks to reduce the effective n processed by the model or the frequency with which inference is invoked at all [15, 25].

Together, these two works define the representational substrate of modern GenAI: Word2Vec established that meaning can be encoded as position in a learned vector space, and the Transformer established that context-sensitive reasoning over such representations can be performed efficiently and at scale. The remainder of this chapter examines the architectural patterns, operational frameworks, and cost optimisation techniques that build upon these foundations in production-grade enterprise systems.

2.2 Large Language Models in Production

The preceding section established the two foundational breakthroughs that made modern language AI possible: distributed word representations and the Transformer architecture. Building upon these primitives, the field witnessed a rapid consolidation around a unified paradigm - the large language model (LLM) - in which ever-larger Transformer-based networks, pre-trained on internet-scale corpora, acquire general-purpose language competence that can be redirected toward downstream tasks with minimal additional supervision. This section characterizes LLMs as a class, traces the methodological evolution from raw pre-training to instruction-following alignment, and examines the architectural and economic implications of deploying such systems at enterprise scale.

2.2.1 Large Language Models: An Overview

A large language model is a neural language model of sufficient parameter count and training data scale that general linguistic and reasoning capabilities emerge as a consequence of the pre-training objective alone, without task-specific architectural modification [4]. Pre-training proceeds via self-supervised next-token prediction over corpora of hundreds of billions to trillions of tokens, optimizing the model to assign high probability to naturally occurring text. The resulting parameter weights encode an implicit representation of syntax, semantics, world knowledge, and rudimentary reasoning chains that can be elicited through natural language prompting.

LLMs are predominantly realized as *decoder-only* Transformer architectures [25], in which the self-attention layers apply a causal mask that restricts each token's attention to its leftward context, preserving the autoregressive generation property. This design choice contrasts with the encoder-only variant typified by BERT [7], which produces bidirectional contextual representations optimized for discriminative tasks. For generative applications - intelligent assistants, automated reasoning, and document synthesis - the decoder-only paradigm dominates, and it is this class of model that underpins the enterprise GenAI systems examined in this dissertation.

A defining empirical finding in the development of LLMs is the existence of *scaling laws*: systematic, power-law relationships between model size, training compute, dataset volume, and downstream performance [4]. Scaling laws imply that performance is largely predictable and improvable by increasing any of the three axes, establishing a framework within which investment in compute translates into measurable capability gains. This property has driven the successive scaling of commercial models and, critically, has also made token consumption a dominant cost variable: larger models process the same input more expensively, directly motivating the cost optimization strategies central to this work.

2.2.2 From Pre-training to Instruction Tuning

Despite the generative competence acquired during pre-training, early LLMs exhibited a systematic misalignment between model behavior and user intent. A model trained solely to predict the next token on a web-scale corpus learns to continue text in statistically plausible ways, but not necessarily in ways that are *helpful*, *honest*, or *harmless* when deployed as an interactive assistant. Ouyang et al. [17] addressed this gap by introducing a three-stage fine-tuning pipeline centered on Reinforcement Learning from Human Feedback (RLHF).

The procedure operates as follows. First, a *supervised fine-tuning* (SFT) phase trains the base model on high-quality demonstrations of desired behavior, produced by human labelers on a curated prompt distribution. Second, a *reward model* is trained to predict human preference scores over pairs of model outputs, internalizing a proxy for human judgment. Third, the SFT model is further optimized against the reward model signal using Proximal Policy Optimization (PPO), an online reinforcement learning algorithm. The resulting InstructGPT models, despite comprising as few as 1.3 billion parameters,

were consistently preferred by human evaluators over the unaligned 175-billion parameter GPT-3 baseline, demonstrating that alignment quality is not strictly a function of scale [17]. This result has profound practical implications: it established RLHF as the standard post-training procedure for production LLMs and confirmed that an aligned smaller model can match or exceed the utility of a larger unaligned one - a relationship that informs the dynamic model selection strategies explored in this dissertation.

2.2.3 Open-Weight Models and the Deployment Spectrum

The commercial LLM landscape is stratified between proprietary, API-accessed models and openly released, self-hostable alternatives. The former class, exemplified by OpenAI's GPT-4 family and Anthropic's Claude series, exposes inference through token-priced cloud APIs; the latter is represented most prominently by Meta AI's Llama series. Touvron et al. [24] introduced Llama 2, a family of models ranging from 7 to 70 billion parameters, pre-trained on two trillion tokens and released under a license permitting commercial use. Fine-tuned variants - Llama 2-Chat - further apply RLHF alignment, achieving performance competitive with closed-source alternatives on standard benchmarks while remaining locally deployable [24].

The existence of capable open-weight models introduces a consequential degree of freedom into enterprise GenAI architectures. Rather than committing uniformly to a single proprietary provider, organizations may route inference dynamically across a portfolio of models differentiated by capability tier, latency characteristics, and per-token cost. Smaller open-weight models, deployed on-premise or on reserved cloud instances, can serve high-volume, lower-complexity queries at substantially reduced marginal cost, while API-accessed frontier models handle tasks requiring peak reasoning performance. This model routing paradigm - often referred to as *LLM cascading* or *dynamic model selection* - is a central cost optimization mechanism evaluated in this dissertation in the context of the Maestro framework's multi-cloud deployment [24, 17].

Together, the progression from raw pre-training through RLHF alignment to the open-weight deployment spectrum defines the operational environment within which enterprise GenAI frameworks must function. The ability to select, combine, and switch between models of varying capability and cost is not merely an architectural convenience but a financial imperative in production systems where token consumption can scale three to five times per client without deliberate optimization [17].

2.3 Retrieval-Augmented Generation

Large language models store factual knowledge implicitly within their parameters, an approach that introduces well-documented limitations: parametric knowledge is frozen at training time, cannot be transparently audited, and tends to produce hallucinations when queries exceed the model's training distribution [12]. Retrieval-Augmented Generation

(RAG) addresses these limitations by coupling a generative model with a dynamic, non-parametric memory that can be queried, updated, and inspected at inference time. The following subsections trace the theoretical foundations of this paradigm, survey its subsequent architectural evolution, and ground the discussion in the dense retrieval machinery that makes scalable semantic search tractable in practice.

2.3.1 Foundational Architecture: Lewis et al. (2020)

The canonical RAG formulation was introduced by Lewis et al. [12], who proposed a general-purpose fine-tuning recipe for generative models endowed with access to an external non-parametric memory. The architecture decomposes into two principal components: (i) a *retriever* $p_\eta(z \mid x)$, implemented as a Dense Passage Retriever (DPR) [10] whose bi-encoder architecture maps both queries and documents into a shared dense vector space via independent BERT encoders, and (ii) a *generator* $p_\theta(y_i \mid x, z, y_{1:i-1})$, implemented as BART-large [11], which conditions its output sequence on the concatenation of the original query x and retrieved passages z.

Formally, the top-K passages are selected by solving a Maximum Inner Product Search (MIPS) problem over a pre-indexed Wikipedia corpus of 21 million 100-word chunks:

$$p_\eta(z \mid x) \;\propto\; \exp\!\big(d(z)^\top q(x)\big)\,, \quad d(z) = \mathrm{BERT}_d(z),\; q(x) = \mathrm{BERT}_q(x). \tag{2.1}$$

Two marginalization strategies over the latent document variable z are proposed. In *RAG-Sequence*, a single retrieved document is assumed responsible for the entire output sequence:

$$p_{\text{RAG-Seq}}(y \mid x) \;\approx\; \sum_{z \in \text{top-}k} p_\eta(z \mid x) \prod_i^N p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{2.2}$$

In *RAG-Token*, a distinct document may be selected at each generation step, enabling the generator to synthesize content from multiple retrieved passages:

$$p_{\text{RAG-Token}}(y \mid x) \;\approx\; \prod_i^N \sum_{z \in \text{top-}k} p_\eta(z \mid x)\, p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{2.3}$$

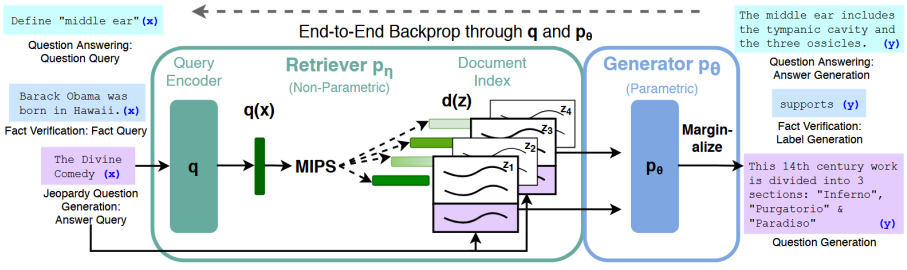

Figure 2.5: End-to-end RAG architecture combining a DPR retriever with a BART-large generator. Adapted from [12].

The retriever and generator are trained end-to-end via minimization of the negative marginal log-likelihood $\sum_j -\log p(y_j \mid x_j)$, without any direct supervision on which documents should be retrieved. Crucially, the document encoder and its FAISS index are kept frozen during fine-tuning; only the query encoder and the BART generator are updated, substantially reducing computational cost.

Empirically, RAG established state-of-the-art results on three open-domain question answering benchmarks-Natural Questions (44.5 EM), WebQuestions (45.2 EM), and CuratedTrec (52.2 EM)-outperforming both purely parametric seq2seq baselines (e.g., T5-11B) and task-specific retrieve-and-extract systems (e.g., DPR-based readers) [12]. For generative tasks, human evaluators rated RAG outputs as more factual than a BART baseline in 42.7% of Jeopardy question generation trials, compared to only 7.1% in which BART was judged superior. An index hot-swapping experiment further demonstrated that updating the non-parametric memory by replacing the document index is sufficient to update the model's world knowledge without any retraining, a property of practical significance for production deployments.

2.3.2 Architectural Evolution: Naive, Advanced, and Modular RAG

Gao et al. [8] systematize the rapid evolution of RAG methods into a three-tier taxonomy, providing a structural lens through which the design of contemporary systems can be evaluated.

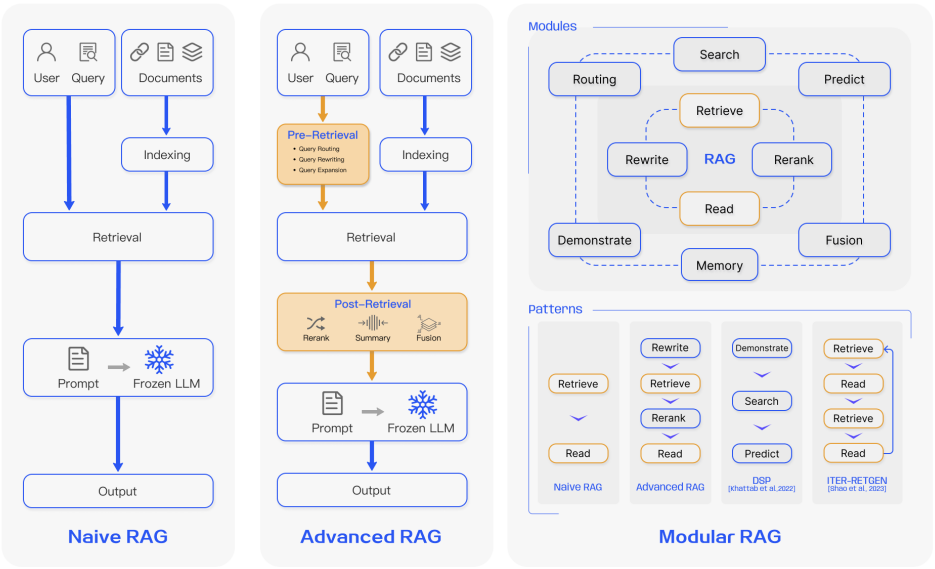

Figure 2.6: Three-tier RAG taxonomy: Naive, Advanced, and Modular RAG. Adapted from [8].

Naive RAG. The earliest post-ChatGPT RAG implementations follow a minimal *Retrieve-then-Read* pipeline consisting of three sequential steps: (1) *indexing*, in which raw documents are chunked, encoded via an embedding model, and stored in a vector database; (2) *retrieval*,

in which the top-K chunks most similar to the user query are identified via cosine similarity over dense embeddings; and (3) *generation*, in which the query and retrieved chunks are concatenated into a prompt supplied to the language model [8]. While effective in simple scenarios, Naive RAG exhibits well-characterized failure modes: low retrieval precision due to semantic gaps between queries and documents, context window saturation from uninformative passages, and hallucinations that arise when the generator defaults to parametric knowledge in the presence of noisy retrieved context.

Advanced RAG. Advanced RAG introduces targeted interventions at both the pre-retrieval and post-retrieval stages to address the precision and coherence limitations of the naive approach [8]. Pre-retrieval strategies include query rewriting, query expansion into sub-questions, and the construction of *Hypothetical Document Embeddings* (HyDE), in which the model first generates a hypothetical answer and then uses its embedding to retrieve semantically closer passages. Post-retrieval strategies include cross-encoder reranking to elevate the most relevant passages and context compression to eliminate redundant or off-topic content prior to generation. Indexing enhancements-such as sliding-window chunking, fine-grained segmentation, and metadata attachment-further improve retrieval recall and temporal awareness. These modifications maintain the linear Retrieve-then-Read structure of Naive RAG while substantially improving end-to-end generation quality.

Modular RAG. Modular RAG supersedes the fixed-pipeline paradigm by decomposing the system into independently composable functional modules: *Search*, *Memory*, *Routing*, *Predict*, and *Task Adapter* [8]. This architecture supports non-sequential execution patterns, including iterative retrieval loops (e.g., ITER-RETGEN [21]), adaptive retrieval triggered by generation uncertainty (e.g., FLARE [9]), and self-reflective retrieval with token-level reflection signals (e.g., Self-RAG [3]). The modular design also enables seamless integration with complementary techniques-fine-tuning the retriever, fine-tuning the generator, or training both collaboratively-without requiring architectural changes to the pipeline. Gao et al. [8] position Modular RAG as the current frontier, noting that its flexibility allows task-specific orchestration patterns that static pipelines cannot accommodate.

A practical implication of this taxonomy is that RAG and fine-tuning are not mutually exclusive. RAG excels in dynamic environments where knowledge must be updated frequently and interpretable provenance is required, whereas fine-tuning is better suited for scenarios demanding stylistic adaptation or the internalization of domain-specific reasoning patterns. In knowledge-intensive tasks, Gao et al. [8] report that RAG consistently outperforms unsupervised fine-tuning, both on knowledge encountered during pre-training and on entirely novel factual information.

2.3.3 Dense Semantic Retrieval: Sentence-BERT

The retrieval component of any RAG pipeline depends critically on the quality and computational tractability of sentence-level embeddings. Vanilla BERT representations are unsuitable for large-scale semantic search: because BERT processes sentence pairs jointly through a cross-attention encoder, identifying the most similar pair in a corpus of n sentences requires $O(n^2)$ inference passes, which Reimers and Gurevych [20] report as approximately 65 hours for $n = 10{,}000$ sentences. Furthermore, averaging BERT token embeddings or using the `[CLS]` token output yields representations that perform worse on semantic textual similarity (STS) tasks than simple GloVe embeddings, owing to BERT's lack of explicit metric learning for sentence-level geometry.

Reimers and Gurevych [20] address these limitations by introducing *Sentence-BERT* (SBERT), a modification of BERT using Siamese and triplet network structures to produce fixed-size, semantically meaningful sentence embeddings amenable to efficient cosine-similarity comparison.

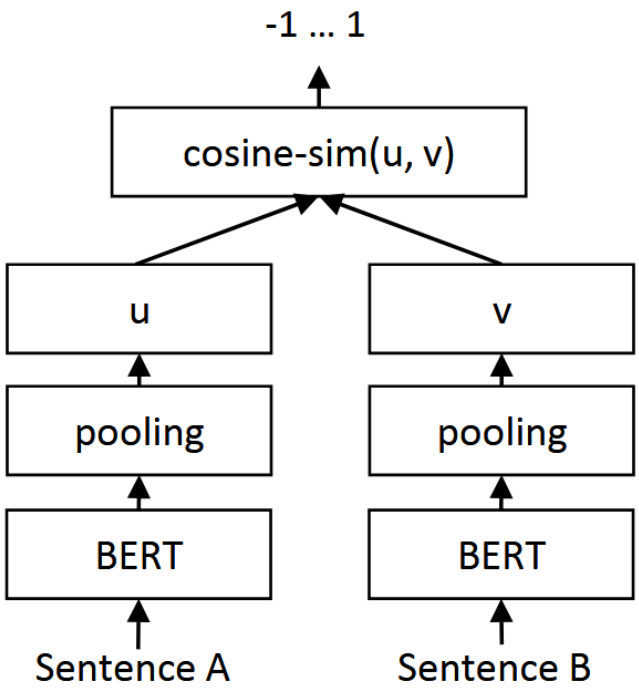

Figure 2.7: SBERT Siamese network architecture with triplet loss objective. Adapted from [20].

The architecture The architecture passes each sentence independently through a shared BERT encoder, then applies mean-pooling over the output token embeddings to obtain a single vector representation $\mathbf{u}$ and $\mathbf{v}$ for a sentence pair. Three training objectives are supported depending on the available supervision signal:

- **Classification objective** (for NLI corpora): a softmax classifier over the concatenation $[\mathbf{u};\ \mathbf{v};\ |\mathbf{u} - \mathbf{v}|]$ trained to predict entailment, contradiction, or neutral labels.

- **Regression objective** (for STS corpora): cosine-similarity between $\mathbf{u}$ and $\mathbf{v}$ optimized with mean-squared error against human-annotated similarity scores.

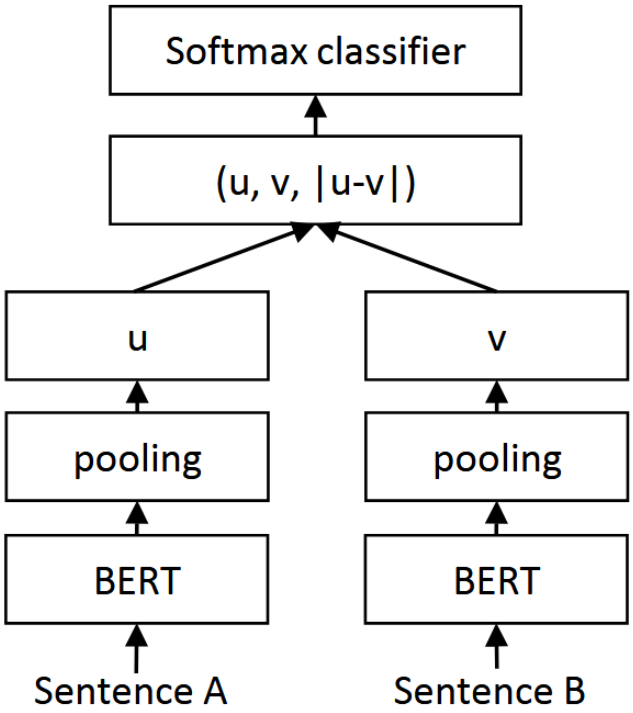

Figure 2.8: Classification and regression training objectives of SBERT. Adapted from [20].

- **Triplet objective**: a margin-based ranking loss that ensures the anchor embedding $\mathbf{a}$ is closer to a positive $\mathbf{p}$ than to a negative $\mathbf{n}$ by a margin ϵ:

$$\mathcal{L}_{\text{triplet}} = \max(\|\mathbf{a} - \mathbf{p}\| - \|\mathbf{a} - \mathbf{n}\| + \epsilon,\ 0)\,. \tag{2.4}$$

The Siamese design decouples encoding from comparison: document embeddings are computed once at indexing time and stored in a vector index, reducing inference at query time to a single forward pass followed by a nearest-neighbor lookup. This reduces the 65-hour pairwise comparison to approximately 5 seconds for a 10,000-sentence corpus, while preserving-or improving upon-BERT's accuracy on STS benchmarks [20]. On the STS tasks evaluated, SBERT achieves an improvement of 11 Spearman correlation points over InferSent and 5 points over Universal Sentence Encoder.

2.3.4 Connection to LlamaIndex

The theoretical contributions reviewed above map directly onto the abstractions provided by LlamaIndex, the framework employed in this work. LlamaIndex operationalizes the RAG pipeline of Lewis et al. [12] through a modular architecture that cleanly separates ingestion, indexing, retrieval, and synthesis [13]. During the *ingestion* stage, `SimpleDirectoryReader` loads raw documents, which are subsequently segmented by `SentenceSplitter` into overlapping chunks-a direct implementation of the chunking strategies discussed by Gao et al. [8] for Advanced RAG. The resulting nodes are encoded via a configurable embedding model (`Settings.embed_model`), which defaults to OpenAI's `text-embedding-3-small` but is readily substituted with any SBERT-compatible model from the HuggingFace ecosystem (e.g., `BAAI/bge-small-en-v1.5`), whose architecture descends directly from the Siamese network paradigm of Reimers and Gurevych [20].

At query time, LlamaIndex's `VectorStoreIndex` retrieves the top-K most relevant nodes via cosine similarity-the same MIPS formulation as DPR in [12]-and passes the retrieved context together with the user query to the configured language model for response synthesis. The framework further exposes advanced retrieval patterns consistent with the Modular RAG taxonomy of Gao et al. [8]: `RouterQueryEngine` implements semantic routing to multiple sub-indexes; post-retrieval reranking modules (e.g., `SentenceTransformerRerank`) apply cross-encoder scoring to improve precision; and iterative retrieval agents can be composed from LlamaIndex's `ReActAgent` infrastructure. In this sense, LlamaIndex provides a production-grade instantiation of the entire RAG design space surveyed by Gao et al. [8], grounded in the dense retrieval foundations established by Lewis et al. [12] and Reimers and Gurevych [20].

2.4 Frameworks, MCP and Maestro

The foundations established in the preceding sections-dense vector representations, attention-based language models, retrieval-augmented generation, and dense semantic retrieval-describe the *components* of a modern GenAI system. What they do not describe is how those components are composed, governed, and operated at production scale. This section addresses that gap across three layers of increasing practical specificity.

2.4.1 LLM Orchestration Frameworks

The emergence of capable large language models created an immediate engineering challenge: how to compose LLMs with external tools, persistent memory, and retrieval systems into coherent, repeatable pipelines. LangChain [5] addressed this challenge by introducing a layered abstraction model built around three core primitives. A *chain* is a composable sequence of calls-to an LLM, a retriever, a tool, or another chain-that can be assembled declaratively and reused across applications. A *tool* is any callable capability exposed to the model with a name, a natural-language description, and a typed input schema;

the LLM uses these descriptions to decide, at runtime, which tool to invoke and with what arguments. *Memory* modules attach conversational context to any chain, enabling stateful interactions across multiple turns. Taken together, these primitives define what has become the dominant orchestration pattern in the field: a model-centric loop in which the LLM acts as a reasoning engine that selects and sequences external capabilities.

The theoretical foundation for this loop was established independently by Yao et al. [26], who proposed the ReAct paradigm-an acronym for *Reason* + *Act*. Rather than treating chain-of-thought reasoning and tool use as separate concerns, ReAct interleaves them into a single iterative cycle: the model emits a *Thought* (an explicit reasoning step), selects an *Action* (a tool call), receives an *Observation* (the tool's output), and repeats until a terminal condition is satisfied. Evaluated on multi-step knowledge-intensive tasks-including HotpotQA, FEVER, ALFWorld, and WebShop-ReAct outperformed both chain-of-thought prompting alone and action-only baselines, demonstrating that the tight coupling of reasoning and acting is essential for robust agent behavior [26]. This loop is the intellectual ancestor of every agent executor in subsequent frameworks, including LangChain's `AgentExecutor`, and it directly informs the orchestration design of the system examined in this work.

2.4.2 Model Context Protocol

A persistent weakness of the orchestration paradigm described above is the absence of a standard interface between agents and the external systems they invoke. In practice, each integration is bespoke: a tool written for LangChain is not directly consumable by a different agent framework, and a function registered for OpenAI's function-calling API is invisible to a Claude-based agent. This $N \times M$ integration problem-N agent frameworks multiplied by M external services-generates duplicated adapter code, inconsistent error handling, and fragile dependency chains that impede both development velocity and maintainability [1].

The Model Context Protocol (MCP), introduced by Anthropic in 2024 [1] and specified in full at the canonical reference documentation [16], addresses this problem by defining an open, transport-agnostic protocol that standardizes how LLM-based hosts expose and consume external capabilities. The protocol organizes participants into three roles. An *MCP Host* is the application embedding the LLM-a development environment, a chat interface, or a custom agent runtime. An *MCP Client* is the component within the host that manages connections to one or more servers. An *MCP Server* is a lightweight process that exposes a bounded set of capabilities over JSON-RPC 2.0, using one of two transport mechanisms: standard I/O for local processes, or HTTP with Server-Sent Events for remote deployments [16]. Capabilities are organized into three first-class primitives: *Tools*-functions that the model may invoke with structured arguments, analogous to LangChain tools but protocol-defined; *Resources*-read-only data artifacts such as file contents, database schemas, or live API responses, exposed for model consumption without side-effects; and

Prompts-parameterized prompt templates that servers can offer for reuse [1, 16].

The distinction between MCP and vendor-specific function calling is substantive. OpenAI's function-calling mechanism, for example, is stateless, scoped to a single model provider, and requires the host application to manage all serialization, discovery, and transport concerns. MCP, by contrast, is stateful at the protocol level, provider-agnostic, and supports dynamic capability discovery: a client can query any MCP server for its current tool and resource catalog without prior knowledge of the server's implementation. The consequence is a reduction from $N \times M$ to $N + M$ integration surface-any MCP-compliant host can interoperate with any MCP-compliant server, regardless of the LLM or framework in use. This interoperability property is directly relevant to multi-cloud enterprise deployments, where the ability to expose heterogeneous data services-across Azure, AWS, and GCP-through a single protocol layer has significant implications for both governance and maintainability.

2.4.3 Enterprise GenAI Frameworks: Maestro in Context

Research prototypes built on frameworks such as LangChain and paradigms such as ReAct operate under a set of assumptions that enterprise production environments do not share. In a production deployment serving multiple tenants across organizational units, correctness and capability are necessary but insufficient conditions. The system must additionally satisfy requirements for identity-based access control, secret management, encryption at rest and in transit, cost observability at the token and model level, multi-tenancy isolation, deployment portability across cloud providers, and operational reliability under variable load-none of which appear as first-class concerns in the orchestration literature reviewed above. Meeting these requirements demands an architectural discipline that goes beyond assembling chains and tools: it requires a governed platform in which each component is independently deployable, swappable without contract changes, and instrumented for both operational monitoring and audit compliance.

The Maestro GenAI Framework, developed by Closer Consulting, represents a practitioner response to precisely this gap [6]. Designed and validated across production deployments in the energy, healthcare, financial, and pharmaceutical sectors, Maestro is a multi-repository, container-native microservices platform whose architecture decomposes the RAG pipeline into independently governed services. The *LLM microservice* acts as a provider-agnostic gateway that routes generation requests to Azure OpenAI (GPT-4o, GPT-4o-mini), OpenAI, Azure AI Services (Mistral-Nemo, DeepSeek-R1), and Google Gemini, exposing a stable contract regardless of the underlying model. The *orchestrator* (chatbot-backend) implements the ReAct-derived retrieval-augmented generation loop-history retrieval, hybrid vector search, per-chunk relevance filtering, prompt assembly, and LLM invocation-as a FastAPI service backed by LangChain. The *vector database service* encapsulates document ingestion, chunking, embedding via `text-embedding-ada-002`, and hybrid

kNN+BM25 retrieval over MongoDB Atlas. The *datastore microservice* provides a provider-agnostic persistence layer over Azure Cosmos DB and MongoDB, covering chat history, feedback, and document metadata. A *guardrails microservice* implements PII detection using Microsoft Presidio for Portuguese-language inputs. Two *frontend surfaces*-a Streamlit chatbot with Microsoft Entra ID authentication and an Angular back-office for prompt and question management-complete the user-facing layer. Infrastructure is managed via OpenTofu targeting Azure, and observability is implemented through OpenTelemetry with Azure Monitor as the backend. In production, the framework has demonstrated measurable impact at scale: one deployment serving over 600 daily active users with 4,000 daily interactions across five embedded LLM models, and another achieving 83% question-answering accuracy over a corpus of 2,907 indexed legal documents [6].

In spite of this production maturity, a systematic analysis of the framework's current implementation reveals a set of structural limitations that constrain its evolution along three dimensions: the absence of a standardized tool-exposure protocol-each microservice is reachable only through bespoke REST endpoints, with no dynamic capability discovery and no interoperability with external agent frameworks; an incomplete cost-observability and optimization layer-token usage and per-model cost are not exported as metrics, and no semantic caching or routing intelligence reduces redundant LLM invocations; and an agent orchestration architecture that relies on hand-crafted orchestrators rather than a composable, reusable agent graph. These three dimensions constitute the research frontier addressed in Section **??**–**??** of this work.

[TODO] Section range above uses placeholder labels; update references when those sections are written.

2.5 Multi-Cloud Computing and Infrastructure as Code

The deployment of production-grade GenAI systems at enterprise scale is not a purely algorithmic challenge. It is equally an infrastructural one. The components surveyed in the preceding sections — large language models, RAG pipelines, embedding models, and orchestration frameworks — must ultimately be provisioned, networked, secured, and operated on physical or virtual compute. The strategic choices made at this infrastructure layer directly influence system cost, resilience, and vendor dependency, each of which is a first-class design concern in the Maestro framework. This section examines the theoretical foundations of cloud computing and multi-cloud strategy, the interoperability and portability challenges that heterogeneous deployments introduce, the research landscape of Infrastructure as Code as a discipline, and the role of declarative provisioning as the operational mechanism that makes reproducible, provider-agnostic infrastructure management tractable at scale.

2.5.1 Foundations of Cloud Computing

[2] identified three properties that fundamentally distinguish cloud computing from prior paradigms of enterprise infrastructure provision: the *illusion of infinite computing resources* available on demand, the *elimination of upfront capital commitments* by transferring hardware acquisition to the provider, and the ability to *pay for resources on a short-term basis* and release them when no longer required. These properties collectively enable a shift from capital expenditure (CapEx), in which organizations maintain underutilized on-premises hardware, to operational expenditure (OpEx), in which compute costs track actual consumption. Armbrust et al. empirically demonstrated that average server utilization in traditional enterprise data centers hovers between 5% and 20%, whereas elastic cloud provisioning allows organizations to closely track real demand and pay accordingly. This cost structure has direct implications for AI-intensive workloads, where inference load is characteristically bursty and difficult to predict in advance: the ability to provision and release compute resources on-demand without upfront commitment is precisely the property that makes multi-model, multi-provider GenAI deployments economically viable.

The same work identified a set of persistent obstacles to full cloud adoption, two of which are particularly relevant to this dissertation. *Data lock-in* arises when an organization's data and application state become tightly coupled to a single provider's proprietary storage and API formats, making migration prohibitively costly and creating asymmetric negotiating power in vendor relationships. *Service availability* refers to the concentration of operational risk in a single provider: outages, degradations, or unilateral policy changes propagate directly to all dependent workloads without recourse. Armbrust et al. identified the distribution of workloads across multiple independent providers as the principal mitigation for both failure modes [2]. This recommendation constitutes the foundational rationale for multi-cloud architectures and directly anticipates the design philosophy adopted in the Maestro framework, which provisions compute and model inference capacity across Azure, AWS, and GCP rather than committing uniformly to any single platform [6].

2.5.2 Interoperability and Portability in Multi-Cloud Environments

Deploying applications across heterogeneous cloud providers introduces a distinct class of engineering challenges that are not resolved by simply replicating resources across platforms. [18] formalized these challenges by distinguishing two complementary but conceptually separate concerns. *Portability* denotes the capacity to transfer an application or workload from one cloud environment to another with bounded effort; *interoperability* denotes the capacity for components deployed on different clouds to communicate and cooperate at runtime. Both properties require explicit architectural investment and neither emerges spontaneously from layering services across providers.

Petcu identified four primary sources of lock-in in multi-cloud deployments: proprietary APIs that expose non-standard resource abstractions; divergent data formats and serialization conventions that impede cross-provider data movement; incompatible service-level agreements that prevent unified governance policies; and asymmetric pricing models that obscure total cost of ownership in comparative analysis. Each source requires a distinct mitigation strategy — respectively, abstraction layers, protocol normalization, contractual harmonization, and unified billing instrumentation [18].

Petcu's taxonomy further classifies multi-cloud deployment strategies along three operational archetypes. A *redundant* strategy designates one provider as active and one or more as standby, achieving resilience through replication at the cost of idle resource expenditure. A *complementary* strategy assigns distinct functional capabilities to distinct providers, exploiting each platform's relative strengths without requiring full workload duplication. A *federated* strategy presents a unified abstraction to the client, with the underlying provider selection transparent to the consuming application [18]. Maestro operates primarily in the complementary mode: AWS Bedrock, Azure OpenAI, and Google Gemini are each exposed through a uniform `llm-microservice` interface that maintains a stable contract regardless of which backend processes a given inference request. This architecture directly instantiates the abstraction-layer pattern advocated by Petcu as the principal mechanism for resolving API heterogeneity across providers, while the stable service contract across providers is precisely what the interoperability axis of her framework prescribes [6].

2.5.3 Infrastructure as Code: Research Landscape and Practice

Infrastructure as Code (IaC) is the practice of expressing the configuration and provisioning of computing infrastructure — virtual machines, networks, storage, and managed services — as machine-readable, version-controlled source files interpreted by a provisioning engine, rather than as manual procedures executed by operators. As cloud adoption accelerated, IaC emerged as the primary mechanism for making infrastructure reproducible, auditable, and amenable to the engineering disciplines of code review and automated testing.

[19] conducted a systematic mapping study of the IaC research landscape, searching six scholarly databases and applying principled inclusion and exclusion criteria to identify 32 peer-reviewed publications directly concerned with IaC. Through qualitative analysis of this corpus, they identified four topics that collectively define the research agenda for the field: (i) *frameworks and tools for IaC*, encompassing the design and evaluation of provisioning engines and domain-specific languages; (ii) *use of IaC in practice*, examining adoption patterns and integration with broader DevOps workflows; (iii) *empirical studies of IaC*, including analyses of defect prevalence, security flaw introduction, and quality metrics in IaC scripts; and (iv) *testing in IaC*, addressing the challenge of validating infrastructure specifications prior to deployment [19]. A notable finding of the mapping

study is that, at the time of publication, empirical research and testing remained substantially underexplored relative to tool-focused contributions — a gap that this dissertation partially addresses by empirically evaluating the cost and operational consequences of IaC-governed multi-cloud GenAI deployments.

Rahman et al. further observed that Ansible, Chef, Puppet, and Terraform were the dominant tools represented in the identified publications, and that research activity on IaC was growing rapidly in proportion to cloud adoption. Their characterization of IaC as a practice that "automatically configures system dependencies and provisions local and remote instances" captures the operational role that IaC plays in Maestro's deployment architecture [19]: the framework's infrastructure layer is managed via OpenTofu [6], the open-source, Linux Foundation-governed fork of HashiCorp Terraform, and thus falls squarely within the tool-focused and empirical categories that Rahman et al. identify as the core research vectors of the field.

OpenTofu operates on a declarative model in which practitioners specify the desired state of infrastructure resources — virtual networks, container registries, managed Kubernetes clusters, secret stores, and monitoring workspaces — and the engine computes a dependency-ordered execution plan to converge the actual state toward the declared target. The provider model maps directly onto the multi-cloud architecture described above: a single OpenTofu configuration references provider plugins for Azure, AWS, and GCP simultaneously, with each resource block scoped to the appropriate provider through namespace qualification. Because the provider abstraction insulates resource declarations from the underlying API differences documented by Petcu [18], operator-facing configuration remains uniform regardless of which cloud hosts a given resource — a property essential for a framework that, by design, spans multiple providers.

Beyond provider abstraction, the declarative IaC approach enables two properties of direct relevance to this dissertation's research objectives. First, it provides *environment reproducibility*: the same configuration that provisions a staging environment in one subscription can instantiate an identical topology in a production subscription or a second cloud region, eliminating configuration drift as a source of environment-specific defects. Second, it establishes a *cost-attribution surface*: because every provisioned resource is declared explicitly in versioned code, cost anomalies can be traced to specific configuration changes through commit history, supporting the FinOps disciplines of budget forecasting and spend attribution that are central to the optimisation objectives of this work [2, 6].

3

Proposed Approach

4

Implementation

4.1 Overview

4.2 Technology Stack

4.3 Framework Implementation

4.3.1 Component A

4.3.2 Component B

4.4 Deployment and Configuration

4.5 Summary

5

Evaluation

6

Conclusion

6.1 Summary of Contributions

6.2 Limitations

6.3 Future Work

6.4 Final Remarks

Bibliography

[1] Anthropic. *Model Context Protocol: Specification*. Tech. rep. Anthropic, 2024. URL: https://www.anthropic.com/news/model-context-protocol (cit. on pp. 14, 15).

[2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia. "A View of Cloud Computing". In: *Communications of the ACM* 53.4 (2010-04), pp. 50–58. DOI: 10.1145/1721654.1721672 (cit. on pp. 17, 19).

[3] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024-05. URL: https://arxiv.org/abs/2310.11511 (cit. on p. 10).

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 1877–1901. DOI: 10.48550/arXiv.2005.14165 (cit. on pp. 4, 6).

[5] H. Chase. *LangChain: Building Applications with Large Language Models*. https://langchain.com. Framework documentation and whitepaper. Accessed: 2026. 2022 (cit. on p. 13).

[6] Closer Consulting. *Maestro GenAI Framework*. Tech. rep. Internal technical documentation and client deployment case studies. Confidential practitioner source. Closer Consulting, 2025. URL: %5Curl%7Bhttps://closerconsultoria.sharepoint.com/:p:/r/sites/GenAIFramework-DEMO/_layouts/15/Doc.aspx?sourcedoc=%7B42094324-FC65-4A0D-9734-58B50056E83C%7D&file=Closer_Maestro_Framework_Closer_Version.pptx&action=edit&mobileredirect=true%7D (cit. on pp. 15–19).

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019-06, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 (cit. on pp. 4, 6).

[8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023-12. DOI: 10.48550/arXiv.2312.10997 (cit. on pp. 9, 10, 13).

[9] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig. "Active Retrieval Augmented Generation". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023-12, pp. 7969–7992. URL: https://arxiv.org/abs/2305.06983 (cit. on p. 10).

[10] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020-11, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550 (cit. on p. 8).

[11] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020-07, pp. 7871–7880. DOI: 10.18653/v1/2020.acl-main.703 (cit. on p. 8).

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 9459–9474. DOI: 10.48550/arXiv.2005.11401 (cit. on pp. 2, 7–9, 13).

[13] J. Liu. *LlamaIndex*. 2022. URL: https://github.com/jerryjliu/llama_index (cit. on p. 13).

[14] J. M. Lourenço. *The `aidisclose` package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: https://ctan.org/pkg/aidisclose (cit. on p. iii).

[15] T. Mikolov, K. Chen, G. Corrado, and J. Dean. *Efficient Estimation of Word Representations in Vector Space*. 2013-09. DOI: 10.48550/arXiv.1301.3781 (cit. on pp. 2, 3, 5).

[16] Model Context Protocol Contributors. *Model Context Protocol — Reference Documentation*. https://modelcontextprotocol.io. Canonical technical specification. Accessed: 2026. 2024 (cit. on pp. 14, 15).

[17] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022-12, pp. 27730–27744. DOI: 10.48550/arXiv.2203.02155 (cit. on pp. 6, 7).

[18] D. Petcu. "Portability and Interoperability between Clouds: Challenges and Case Study". In: *Towards a Service-Based Internet*. Lecture Notes in Computer Science 8135 (2013), pp. 62–74. DOI: 10.1007/978-3-642-45015-0_6 (cit. on pp. 17–19).

[19] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams. "A Systematic Mapping Study of Infrastructure as Code Research". In: *Information and Software Technology* 108 (2019-04), pp. 65–77. DOI: 10.1016/j.infsof.2018.12.004 (cit. on pp. 18, 19).

[20] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019-11, pp. 3982–3992. DOI: 10.18653/v1/D19-1410 (cit. on pp. 11–13).

[21] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen. "Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy". In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023-12, pp. 9248–9274. URL: https://arxiv.org/abs/2305.15294 (cit. on p. 10).

[22] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. "GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing". In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).

[23] The LaTeX Project team. *LaTeX - A document preparation system*. 1985. URL: https://www.latex-project.org (visited on 2025-12-29) (cit. on p. iii).

[24] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023-07. DOI: 10.48550/arXiv.2307.09288 (cit. on p. 7).

[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017-12. DOI: 10.48550/arXiv.1706.03762 (cit. on pp. 4–6).

[26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: https://arxiv.org/abs/2210.03629 (cit. on p. 14).

A

Appendix Title

B

Appendix Title